LPII - Pesquisa Baseada em Atributos de Várias Entidades

1 – Construindo a JanelaCadastroAvaliações

A seguir, os enumerados, atributos e referências da entidade Avaliação:

```
    public enum Classificação {péssima, fraca, regular, boa, excelente};
    public enum Preferência { entre_dez_mais, entre_dez_menos };

    private int sequencial;
    private Amigo amigo;
    private Filme filme;
    private Classificação classificação_trama, classificação_direção, classificação_atuação;
    private Preferência preferência;
    private Timestamp data_hora;
```

A seguir, o script SQL da tabela Avaliações:

```
CREATE TABLE Avaliações (
   Sequencial INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
   AmigoId VARCHAR(50) NOT NULL,
   FilmeId INT NOT NULL,
   ClassificaçãoTrama INT NOT NULL,
   ClassificaçãoDireção INT NOT NULL,
   ClassificaçãoAtuação INT NOT NULL,
   Preferência INT,
   DataHora TIMESTAMP NOT NULL,
   FOREIGN KEY (AmigoId) REFERENCES Amigos(Nome),
   FOREIGN KEY (FilmeId) REFERENCES Filmes(Sequencial));
```

Observe, que na entidade Avaliação o atributo data_hora é do tipo Timestamp. De forma similar, no script para criação da tabela Avaliações, o tipo é definido como TIMESTAMP. Esse tipo de dado permite que consultas SQL recuperem registros da tabela tomando como base uma determinada data.

Os componentes gráficos utilizados para construir a JanelaCadastroAvaliações, já foram utilizados nos tutoriais anteriores e, portanto, sua configuração já é conhecida.

A seguir, as referências e o construtor da JanelaCadastroAvaliações:

public class JanelaCadastroAvaliações extends javax.swing.JFrame {

```
    ControladorCadastroAvaliações controlador;
    DefaultListModel modelo_lista_avaliações;
    Amigo[] amigos_cadastrados;
    Filme[] filmes_cadastrados;

    public JanelaCadastroAvaliações(ControladorCadastroAvaliações controlador) {
        this.controlador = controlador;
        amigos_cadastrados = Amigo.getVisões();
        filmes_cadastrados = Filme.getVisões();
        initComponents();
        inicializarListaAvaliações();
        limparCampos(null);
    }
```

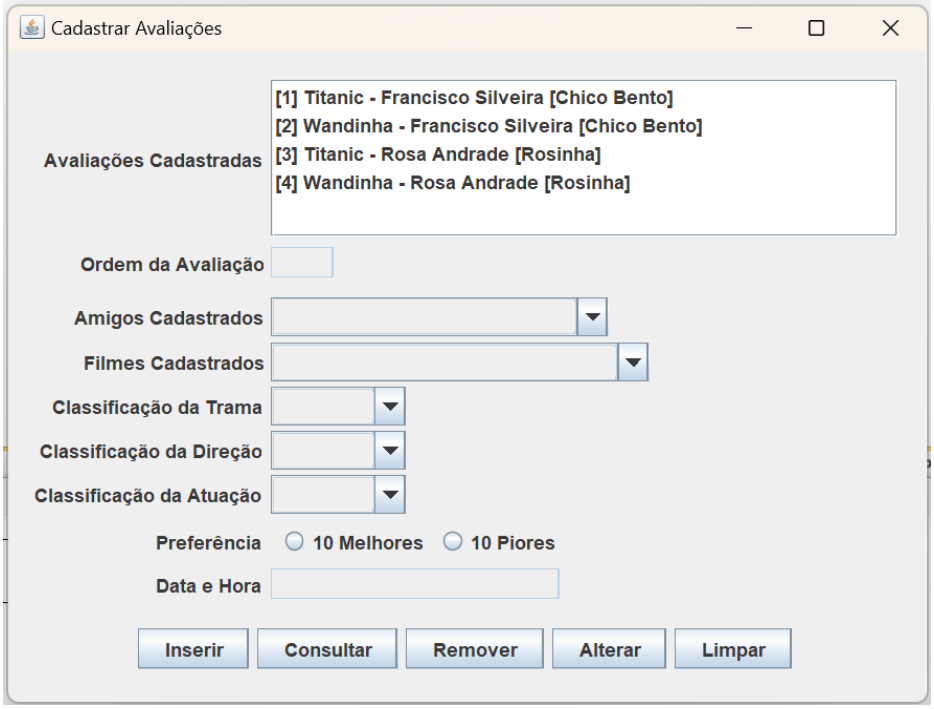
O método auxiliar inicializarListaAvaliações é semelhante ao método utilizado na JanelaCadastroFilmes. Esse método executa o método estático getVisões, da entidade Avaliação.

```
private void inicializarListaAvaliações() {
    modelo_lista_avaliações = (DefaultListModel) avaliações_cadastradasList.getModel();
    Avaliação[] visões = Avaliação.getVisões();
    for (Avaliação visão : visões) {
        modelo_lista_avaliações.addElement(visão);
    }
}

public static Avaliação[] getVisões () {
    String sql = "SELECT Sequencial, AmigoId, FilmeId FROM Avaliações";
    ResultSet lista_resultados = null;
    ArrayList<Avaliação> visões = new ArrayList();
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            visões.add(new Avaliação (lista_resultados.getInt("Sequencial"),
                Amigo.buscarAmigo(lista_resultados.getString("AmigoId")).getVisão(),
                Filme.buscarFilme(lista_resultados.getInt("FilmeId")).getVisão()));
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    return visões.toArray(new Avaliação[visões.size()]);
}
```

Neste tutorial, em vez de criar um método auxiliar para limpar os campos da Avaliação, é chamado o próprio tratador de eventos (ilustrado na seção 1.5), vinculado ao botão de comandos Limpar, passando null como argumento, dado que neste caso, a chamada ocorre a partir do construtor e não de um evento gerado pelo usuário.

A seguir, é ilustrada a JanelaCadastroAvaliações.



Observe que a janela utiliza um componente List para mostrar as avaliações cadastradas, e componentes ComboBox para selecionar um amigo, um filme e os atributos da Avaliação associados a enumerados com vários valores.

1.1 - O Tratamento de Evento para Inserir uma Avaliação

O tratador de eventos inserirAvaliação da JanelaCadastroAvaliações é ilustrado a seguir.

```
private void inserirAvaliação(java.awt.event.ActionEvent evt) {
    Avaliação avaliação = obtémAvaliaçãoInformada();
    String mensagem_erro = null;
    if (avaliação != null) mensagem_erro = controlador.inserirAvaliação(avaliação);
    else mensagem_erro = "Algum atributo da avaliação não foi informado";
    if (mensagem_erro == null) {
        int sequencial = Avaliação.últimoSequencial();
        avaliação.setSequencial(sequencial);
        modelo_lista_avaliações.addElement(avaliação.getVisão());
        avaliações_cadastradasList.setSelectedIndex(modelo_lista_avaliações.size() - 1);
        sequencialTextField.setText("" + sequencial);
        data_horaTextField.setText(Avaliação.formatarDataHora
            (avaliação.getDataHora().toString()));
    } else informarErro (mensagem_erro);
}
```

O método auxiliar obtémAvaliaçãoInformada é ilustrado a seguir. Observe que o objeto da Avaliação é criado referenciando as visões de Amigo e Filme.

```
private Avaliação obtémAvaliaçãoInformada() {
    String sequencial_str = sequencialTextField.getText();
    int sequencial = 0;
    if (!sequencial_str.isEmpty()) sequencial = Integer.parseInt(sequencial_str);
    Amigo visão_amigo = (Amigo) amigos_cadastradosComboBox.getSelectedItem();
    if (visão_amigo == null) return null;
    Filme visão_filme = (Filme) filmes_cadastradosComboBox.getSelectedItem();
    if (visão_filme == null) return null;
    Classificação classificação_trama = null;
    if (classificação_tramaComboBox.getSelectedItem() != null)
        classificação_trama = (Classificação) classificação_tramaComboBox.getSelectedItem();
    else return null;
    Classificação classificação_direção = null;
    if (classificação_direçãoComboBox.getSelectedItem() != null)
        classificação_direção = (Classificação) classificação_direçãoComboBox.getSelectedItem();
    else return null;
    Classificação classificação_atuação = null;
    if (classificação_atuaçãoComboBox.getSelectedItem() != null)
        classificação_atuação = (Classificação) classificação_atuaçãoComboBox.getSelectedItem();
    else return null;
    Preferência preferência = null;
    if (preferênciaButtonGroup.getSelection() != null)
        preferência = Preferência.values()[preferênciaButtonGroup.getSelection().getMnemonic()];
    Timestamp data_hora = new Timestamp(Calendar.getInstance().getTimeInMillis());
    return new Avaliação(sequencial, visão_amigo, visão_filme, classificação_trama,
        classificação_direção, classificação_atuação, preferência, data_hora);
}
```

O método inserirAvaliação do ControladorCadastroAvaliações:

```
public String inserirAvaliação (Avaliação avaliação) {
    if (!Avaliação.existeAvaliação (avaliação.getAmigo().getNome(),
        avaliação.getFilme().getSequencial())) {
        return Avaliação.inserirAvaliação (avaliação);
    } else return "Avaliação já cadastrada";
}
```

A seguir, o método estático existeAvaliação da entidade Avaliação:

```
public static boolean existeAvaliação (String chave_amigo, int chave_filme) {
    String sql = "SELECT Sequencial FROM Avaliações WHERE AmigoId = ? AND FilmeId = ?";
    ResultSet lista_resultados = null;
    boolean existe = false;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setString(1, chave_amigo);
        comando.setInt(2, chave_filme);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            existe = true;
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
    }
    return existe;
}
```

O método estático inserirAvaliação, da entidade Avaliação, utiliza as chaves das visões de Amigo e Filme para compor o registro da avaliação sendo inserida no BD:

```
public static String inserirAvaliação (Avaliação avaliação) {
    String sql = "INSERT INTO Avaliações (AmigoId, FilmeId, ClassificaçãoTrama,"
       + " ClassificaçãoDireção, ClassificaçãoAtuação, Preferência, DataHora)"
       + " VALUES (?, ?, ?, ?, ?, ?, ?)";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setString(1, avaliação.getAmigo().getNome());
        comando.setInt(2, avaliação.getFilme().getSequencial());
        comando.setInt(3, avaliação.getClassificaçãoTrama().ordinal());
        comando.setInt(4, avaliação.getClassificaçãoDireção().ordinal());
        comando.setInt(5, avaliação.getClassificaçãoAtuação().ordinal());
        int índice_preferência = -1;
        if (avaliação.getPreferência() != null)
            índice_preferência = avaliação.getPreferência().ordinal();
        comando.setInt(6, índice_preferência);
        comando.setTimestamp(7, avaliação.getDataHora());
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
        return "Erro na Inserção da Avaliação no BD";
    }
}
```

1.2 - O Tratamento de Evento para Alterar uma Avaliação

O tratador de eventos alterarAvaliação da JanelaCadastroAvaliações é ilustrado a seguir. Observe, que para a entidade associadora Avaliação, na qual são mostradas no componente avaliações_cadastradasList somente o nome do Amigo e o título do Filme, não é necessário alterar o texto mostrado, dado que a avaliação é caracterizada pela associação de um amigo e de um filme, e mudança de um deles descaracterizaria a avaliação. Portanto, neste evento, ocorrem somente alterações dos atributos da Avaliação, mas as suas referências não podem ser alteradas.

```
private void alterarAvaliação(java.awt.event.ActionEvent evt) {
    Avaliação avaliação = obtémAvaliaçãoInformada();
    String mensagem_erro = null;
    if (avaliação != null) mensagem_erro = controlador.alterarAvaliação(avaliação);
    else mensagem_erro = "Algum atributo da avaliação não foi informado";
    if (mensagem_erro != null) informarErro (mensagem_erro);
}
```

No método alterarAvaliação do ControladorCadastroAvaliações, as chaves das visões selecionadas pelo usuário para amigo e filme, são comparadas com as respectivas chaves dos objetos de amigo e filme da avaliação cadastrada. De forma similar ao que foi comentado no parágrafo anterior, o controlador só permite a alteração da avaliação se encontrar uma avaliação caracterizada pelo amigo e filme.

```
public String alterarAvaliação(Avaliação avaliação_informada) {
    Avaliação avaliação_cadastradas = Avaliação.buscarAvaliação
         (avaliação_informada.getSequencial());
    if ((avaliação_informada.getAmigo().getNome().equals
        (avaliação_cadastradas.getAmigo().getNome()))
        && (avaliação_informada.getFilme().getSequencial()
            == avaliação_cadastradas.getFilme().getSequencial())) {
        return Avaliação.alterarAvaliação (avaliação_informada);
    } else return "Alteração não permitida : chave do amigo e/ou do filme foram alteradas";
}
```

A seguir, o método estático alterarAvaliação da entidade Avaliação:

```
public static String alterarAvaliação (Avaliação avaliação) {
    String sql = "UPDATE Avaliações SET ClassificaçãoTrama = ?, ClassificaçãoDireção = ?,"
            + " ClassificaçãoAtuação = ?, Preferência = ? WHERE Sequencial = ?";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, avaliação.getClassificaçãoTrama().ordinal());
        comando.setInt(2, avaliação.getClassificaçãoDireção().ordinal());
        comando.setInt(3, avaliação.getClassificaçãoAtuação().ordinal());
        int índice_preferência = -1;
        if (avaliação.getPreferência() != null)
             índice_preferência = avaliação.getPreferência().ordinal();
        comando.setInt(4, índice_preferência);
        comando.setInt(5, avaliação.getSequencial());
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
        return "Erro na Alteração da Avaliação no BD";
    }
}
```

1.3 - O Tratamento de Evento para Consultar uma Avaliação

O tratador de eventos consultarAvaliação da JanelaCadastroAvaliações é ilustrado a seguir.

```
private void consultarAvaliação(java.awt.event.ActionEvent evt) {
    Avaliação visão_avaliação = (Avaliação) avaliações_cadastradasList.getSelectedValue();
    Avaliação avaliação = null;
    String mensagem_erro = null;
    if (visão_avaliação != null) {
        avaliação = Avaliação.buscarAvaliação(visão_avaliação.getSequencial());
        if (avaliação == null) mensagem_erro = "Avaliação não cadastrada";
    } else mensagem_erro = "Nenhuma avaliação selecionada";
    if (mensagem_erro == null) {
        sequencialTextField.setText(avaliação.getSequencial() + "");
        amigos_cadastradosComboBox.setSelectedItem(getVisãoAmigoSelecionada(avaliação));
        filmes_cadastradosComboBox.setSelectedItem(getVisãoFilmeSelecionada(avaliação));
        classificação_tramaComboBox.setSelectedItem(avaliação.getClassificaçãoTrama());
        classificação_direçãoComboBox.setSelectedItem(avaliação.getClassificaçãoDireção());
        classificação_atuaçãoComboBox.setSelectedItem(avaliação.getClassificaçãoAtuação());
        selecionePreferênciaRadioButton(avaliação.getPreferência());
        data_horaTextField.setText(Avaliação.formatarDataHora
            (avaliação.getDataHora().toString()));
    } else informarErro (mensagem_erro);
}
```

O método estático formatarDataHora, da entidade Avaliação, é ilustrado a seguir. Este método é necessário para converter :

- o formato recebido do banco de dados: 2024-08-28 10:15:0001;
- para o formato a ser visualizado na JanelaCadastroAvaliações: 29/08/2024 10:15hs.

```
public static String formatarDataHora(String data_hora_bd) {
    String[] data_hora = data_hora_bd.split(" ");
    String[] ano_mês_dia = data_hora[0].split("-");
    String[] hora_minuto_resto = data_hora[1].split(":");
    String data_hora_formatada = ano_mês_dia[2] + "/" + ano_mês_dia[1] + "/" + ano_mês_dia[0]
        + " " + hora_minuto_resto[0]+ ":" + hora_minuto_resto[1] + "hs";
    return data_hora_formatada;
}
```

O tratador de eventos consultarAvaliação executa o método estático buscarAvaliação da entidade Avaliação.

```
public static Avaliação buscarAvaliação (int sequencial) {
    String sql = "SELECT * FROM Avaliações WHERE Sequencial = ?";
    ResultSet lista_resultados = null;
    Avaliação avaliação = null;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            Preferência preferência = null;
            if (lista_resultados.getInt("Preferência") > -1)
                preferência = Preferência.values()[lista_resultados.getInt("Preferência")];
            avaliação = new Avaliação (sequencial,
                Amigo.buscarAmigo(lista_resultados.getString("AmigoId")),
                Filme.buscarFilme(lista_resultados.getInt("FilmeId")),
                Classificação.values()[lista_resultados.getInt("ClassificaçãoTrama")],
                Classificação.values()[lista_resultados.getInt("ClassificaçãoDireção")],
                Classificação.values()[lista_resultados.getInt("ClassificaçãoAtuação")],
                preferência,
                lista_resultados.getTimestamp("DataHora"));
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
        avaliação = null;
    }
    return avaliação;
}
```

1.4 - O Tratamento de Evento para Remover uma Avaliação

O tratador de eventos removerAvaliação, da JanelaCadastroAvaliações, é semelhante ao método equivalente da JanelaCadastroFilmes.

O método removerAvaliação do ControladorCadastroAvaliações é ilustrado a seguir.

```
public String removerAvaliação (int sequencial) {
    if (Avaliação.existeAvaliação(sequencial)) return Avaliação.removerAvaliação (sequencial);
    else return "Sequencial não corresponde a nenhuma reserva cadastrada";
}
```

O método estático existeAvaliação da entidade Avaliação, chamado pelo método removerAvaliação, é diferente do método de mesmo nome chamado pelo método inserirAvaliação. Ao inserir uma avaliação, não é possível passar o seu sequencial, dado que ele ainda não foi criado pelo banco de dados. Já na remoção, o sequencial pode ser utilizado.

```
public static boolean existeAvaliação (int sequencial) {
    String sql = "SELECT COUNT(Sequencial) FROM Avaliações WHERE Sequencial = ?";
    ResultSet lista_resultados = null;
    boolean existe = false;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            existe = true;
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
    }
    return existe;
}
```

1.5 - O Tratamento de Evento para Limpar os Dados da Avaliação

O tratador de eventos do botão de comandos Limpar, o método limparCampos é chamado ilustrado a seguir.

```
private void limparCampos(java.awt.event.ActionEvent evt) {
    sequencialTextField.setText("");
    amigos_cadastradosComboBox.setSelectedIndex(-1);
    filmes_cadastradosComboBox.setSelectedIndex(-1);
    classificação_tramaComboBox.setSelectedIndex(-1);
    classificação_direçãoComboBox.setSelectedIndex(-1);
    classificação_atuaçãoComboBox.setSelectedIndex(-1);
    preferênciaButtonGroup.clearSelection();
    data_horaTextField.setText("");
}
```

2 – Construindo a JanelaPesquisaAvaliações

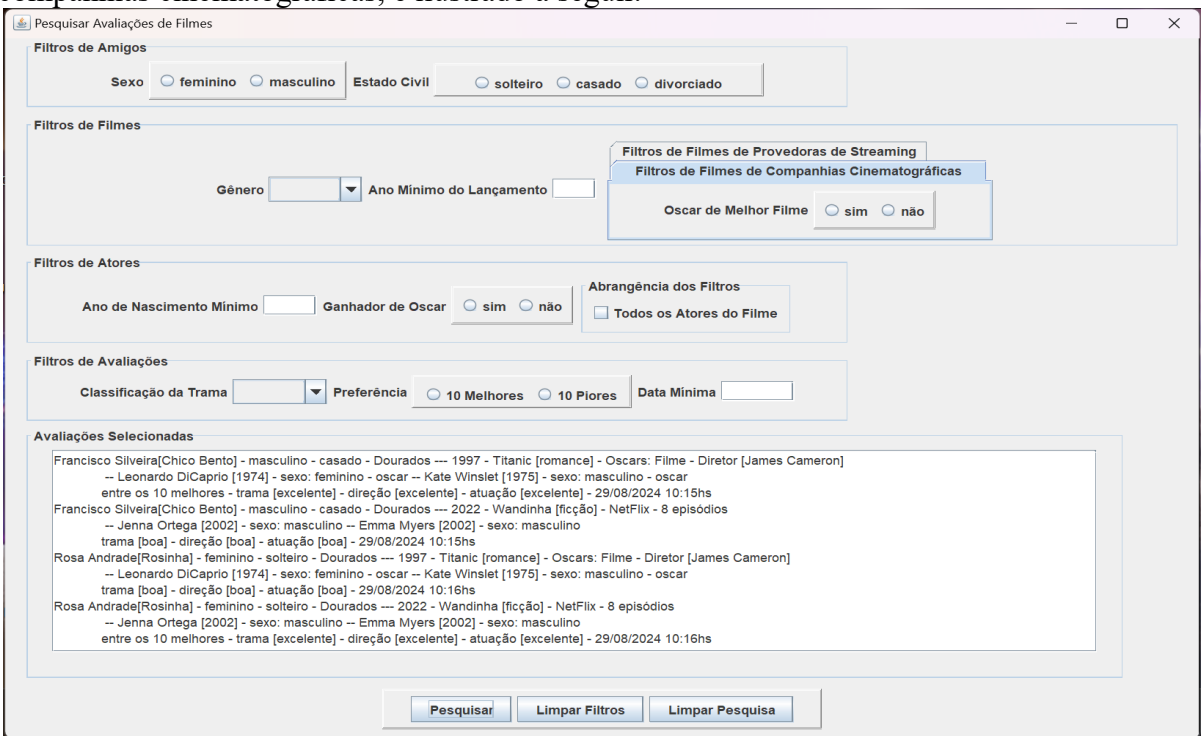
De forma semelhante à JanelaCadastroFilmes, a JanelaPesquisaAvaliações utiliza um painel com abas e paineis auxiliares para construir os formulários das suas abas, com filtros baseados nas subclasses da entidade Filme. As referências e o construtor da JanelaPesquisaAvaliações :

```
PainelFiltrosFilmeCompanhiaCinematográfica filtros_filme_companhia_cinematográficaPainel;
PainelFiltrosFilmeProvedoraStreaming filtros_filme_provedora_streamingPainel;

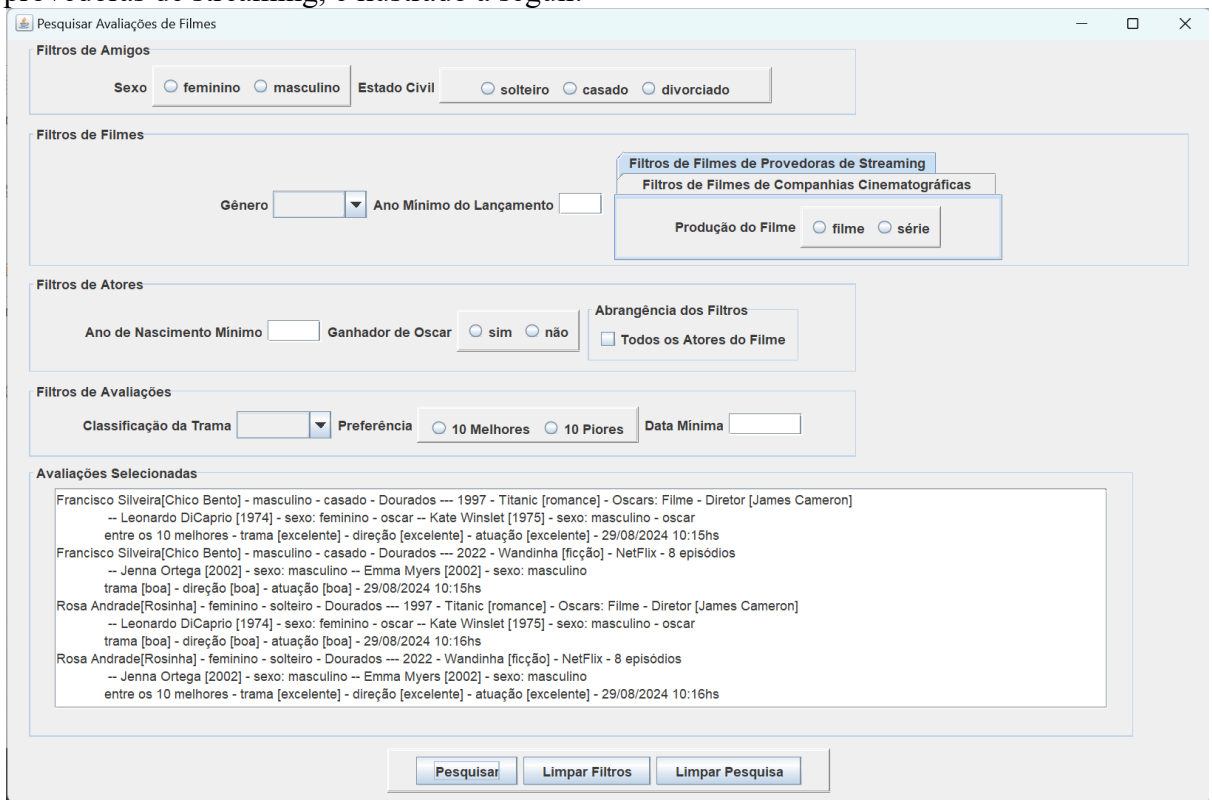

public JanelaPesquisaAvaliações() {
    initComponents();
    filtros_filme_companhia_cinematográficaPainel
        = new PainelFiltrosFilmeCompanhiaCinematográfica();
    filtros_filme_provedora_streamingPainel = new PainelFiltrosFilmeProvedoraStreaming();
    filtros_específicos_filmeTabbedPane.addTab
        ("Filtros de Filmes de Companhias Cinematográficas",
    filtros_filme_companhia_cinematográficaPainel);
    filtros_específicos_filmeTabbedPane.addTab("Filtros de Filmes de Provedoras de Streaming",
    filtros_filme_provedora_streamingPainel);
    limparFiltros(null);
}
```

A JanelaPesquisaAvaliações não utiliza controlador para decidir a respeito das operação porque suporta somente pesquisa e, portanto, não há nenhuma decisão a ser tomada pelo controlador, a respeito de criação, remoção ou alteração de registros nas tabelas do banco de dados.

O layout da JanelaPesquisaAvaliações, com a pesquisa inicial com foco na aba de filtros para filmes de companhias cinematográficas, é ilustrado a seguir.



O layout da JanelaPesquisaAvaliações, com a pesquisa inicial com foco na aba de filtros para filmes de provedoras de streaming, é ilustrado a seguir.



Filtros estão associados a atributos (propriedades ou características) de uma dada entidade. Neste tutorial, foram implementados mais de um filtro para cada entidade primária (que tem atributos): Avaliação, Amigo, Filme e Ator. Observe que não foram definidos filtros para a entidade secundária Atuação, dado que ela é criada somente para representar um relacionamento [n:n] no banco de dados e, portanto, tem duas referências (filme, ator), mas nenhum atributo. Na Entrega 4, será solicitado somente um filtro para cada uma das entidades.

No layout da JanelaPesquisaAvaliações, os filtros de cada entidade são agrupados em painéis. É utilizado um painel para agrupar os filtros relacionados com a entidade Amigo, outro para os filtros relacionados com a entidade Filme e suas subclasses, outro para os filtros da entidade Ator e outro para os filtros da entidade Avaliação. O painel com os filtros da entidade Filme é composto de dois painéis internos: um para os filtros da entidade Filme e outro com duas abas, sendo que a cada aba é associado um painel externo referente aos filtros associados a uma dada subclasse da entidade Filme. Os resultados da pesquisa, são mostrados em uma TextArea, encapsulada por um ScrollPane. Tanto o ScrollPane, quanto os botões de comando, também são encapsulados por painéis.

No filme avaliado por um amigo, podem atuar vários atores. Para caracterizar se todos os atores do filme devem atender aos filtros, ou se basta que um dos atores do filme atenda os filtros, foi incluído, no painel dos filtros dos atores, o CheckBox rotulado pelo string Todos os Atores do Filme. Para separar este flag dos filtros dos atores, o CheckBox foi encapsulado por um painel intitulado: Abrangência dos Filtros.

Quando nenhum filtro, associado a uma das subclasses de filme, é selecionado (ou informado), avaliações envolvendo filmes das duas subclasses podem ser listadas na pesquisa. No entanto, quando um filtro relacionado a uma subclasse de filme é selecionado (ou informado), somente avaliações envolvendo esta subclasse do filme podem ser listadas na pesquisa.

2.1 - O Tratamento de Evento para Pesquisar Avaliações

O método tratador de eventos pesquisarAvaliações é ilustrado a seguir.

```
private void pesquisarAvaliações(java.awt.event.ActionEvent evt) {
    Sexo sexo_amigo = null;
    if (sexo_amigoButtonGroup.getSelection() != null)
        sexo_amigo = Sexo.values()[sexo_amigoButtonGroup.getSelection().getMnemonic()];
    EstadoCivil estado_civil_amigo = null;
    if (estado_civil_amigoButtonGroup.getSelection() != null)
        estado_civil_amigo = EstadoCivil.values()
            [estado_civil_amigoButtonGroup.getSelection().getMnemonic()];
    Gênero gênero_filme = null;
    if (gêneros_filmesComboBox.getSelectedItem() != null)
        gênero_filme = (Gênero) gêneros_filmesComboBox.getSelectedItem();
    int ano_filme = -1;
    if (!ano_mínimo_filmeTextField.getText().isEmpty())
         ano_filme = Integer.parseInt(ano_mínimo_filmeTextField.getText());
    int ano_nascimento_mínimo_ator = -1;
    if (!ano_nascimento_mínimo_atorTextField.getText().isEmpty())
         ano_nascimento_mínimo_ator = Integer.parseInt
              (ano_nascimento_mínimo_atorTextField.getText());
    char ator_ganhador_oscar = getAtorGanhadorOscar();
    boolean todos_atores_filme = todos_atores_filmeCheckBox.isSelected();
    char oscar_melhor_filme = 'X';
    Produção produção_filme = null;
    int índice_aba_secionada = filtros_específicos_filmeTabbedPane.getSelectedIndex();
    if (índice_aba_secionada == 0)
        oscar_melhor_filme
            = filtros_filme_companhia_cinematográficaPainel.getOscarMelhorFilme();
    else if (índice_aba_secionada == 1)
        produção_filme = filtros_filme_provedora_streamingPainel.getSelectedProdução();
    Classificação classificação_trama
        = (Classificação) avaliação_classificação_tramaComboBox.getSelectedItem();
    Preferência preferência = null;
    if (preferênciaButtonGroup.getSelection() != null)
        preferência = Preferência.values()[preferênciaButtonGroup.getSelection().getMnemonic()];
    Timestamp data_mínima = getDataMínima();
    ArrayList<Avaliação> avaliações = Avaliação.pesquisarAvaliações
       (sexo_amigo, estado_civil_amigo,
        gênero_filme, ano_mínimo_filme, oscar_melhor_filme, produção_filme,
        ano_nascimento_mínimo_ator, ator_ganhador_oscar, todos_atores_filme,
        classificação_trama, preferência, data_mínima);
    mostrarAvaliaçõesSelecionadas(avaliações);
}
```

O método auxiliar getAtorGanhadorOscar é utilizado para ler o grupo de botões de radio, que serve para o usuário informar se deseja um ator ganhador de oscar. O filtro ator_ganhador_oscar, embora esteja relacionado com o atributo booleano de mesmo nome na entidade Filme, é declarado como char. Um filtro não pode ser representado por uma variável do tipo boolean, que só pode representar sim ou não, mas não pode representar qualquer, sinalizando filtro não informado. Uma boa solução é transformar um filtro do tipo boolean em tipo char, para o qual é possível representar: 'T' para true, 'F' para false e 'X' para qualquer. Pelo mesmo motivo esse tipo de filtro não pode ser representado pelo componente CheckBox. Uma boa alternativa é utilizar dois botões de rádio intitulados: sim e não. A não seleção dos botões de radio indica a opção qualquer para o filtro.

```
private char getAtorGanhadorOscar() {
    char ator_ganhador_oscar = 'X';
    if (ator_ganhador_oscarButtonGroup.getSelection() != null) {
        switch(ator_ganhador_oscarButtonGroup.getSelection().getMnemonic()) {
            case 0: ator_ganhador_oscar = 'T'; break;
            case 1: ator_ganhador_oscar = 'F';
        }
    }
    return ator_ganhador_oscar;
}
```

A janela PainelFiltrosFilmeCompanhiaCinematográfica provê os seguintes métodos:

```
public char getOscarMelhorFilme() {
    char oscar_melhor_filme = 'X';
    if (oscar_melhor_filmeButtonGroup.getSelection() != null) {
        switch(oscar_melhor_filmeButtonGroup.getSelection().getMnemonic()) {
            case 0: oscar_melhor_filme = 'T'; break;
            case 1: oscar_melhor_filme = 'F';
        }
    }
    return oscar_melhor_filme;
}

public void limparFiltros() { oscar_melhor_filmeButtonGroup.clearSelection(); }
```

De forma semelhante, ao observado anteriormente para ator_ganhador_oscar, o filtro oscar_melhor_filme é declarado como char, e implementado na interface como um grupo de botões de radio.

A janela PainelFiltrosFilmeProvedoraStreaming provê os seguintes métodos:

```
public FilmeProvedoraStreaming.Produção getSelectedProdução() {
    FilmeProvedoraStreaming.Produção produção = null;
    if (produção_filmeButtonGroup.getSelection() != null)
        produção = FilmeProvedoraStreaming.Produção.values()
            [produção_filmeButtonGroup.getSelection().getMnemonic()];
    return produção;
}

public void limparFiltros() {
    produção_filmeButtonGroup.clearSelection();
}
```

O método auxiliar getDataMínima, que converte o formato digitado pelo usuário (29/08/20024 10:15hs) para um valor do tipo Timestamp a ser utilizado como atributo para método pesquisarAvaliações , é ilustrado a seguir,.

```
private Timestamp getDataMínima() {
    Timestamp data_mínima = null;
    String data_mínima_str = data_mínima_avaliaçãoTextField.getText();
    if (!data_mínima_str.isEmpty()) {
        String[] data_mínima_partes = data_mínima_str.toString().split("/");
        String dia = data_mínima_partes[0];
        String mês = data_mínima_partes[1];
        String ano = data_mínima_partes[2];
        if ((dia.length() == 2) && (mês.length() == 2) && (ano.length() == 4)) {
            data_mínima_str = ano + "-" + mês + "-" + dia + " 00:00:00";
            data_mínima = Timestamp.valueOf(data_mínima_str);
        }
    }
    return data_mínima;
}
```

O método auxiliar mostrarAvaliaçõesSelecionadas é utilizado para gerar um string com todas as avaliações selecionadas e apendar no texto existente na TextArea, relacionado com as pesquisas anteriores.

```
private void mostrarAvaliaçõesSelecionadas(ArrayList<Avaliação> avaliações) {
    boolean primeira_avaliação = true;
    for (Avaliação avaliação : avaliações) {
        if (primeira_avaliação) {
            pesquisasTextArea.append(avaliação.toStringFull());
            primeira_avaliação = false;
        } else pesquisasTextArea.append("\n" + avaliação.toStringFull());
    }
}
```

Pelo fato de que os métodos toString das entidades concatenam apenas os atributos essenciais (visão) para visualização de itens cadastrados no ComboBox ou List, são criados métodos toStringFull nas entidades para encorporar os vários atributos de cada entidade, que deverão ser visualizados na lista de avaliações selecionadas, mostradas no TextArea. A seguir, o método toStringFull da entidade Avaliação, que utiliza o método auxiliar formatarDataHora, ilustrado na seção 1.3.

```
public String toStringFull() {
    String str = amigo.toStringFull() + " --- " + filme.toStringFull() + "\n             ";
    Ator[] atores_filme = Atuação.buscarAtoresFilme(filme.getSequencial());
    for (Ator ator : atores_filme) str += " -- " + ator.toStringFull();
    str += "\n             ";
    if (preferência != null) {
        switch(preferência) {
            case entre_dez_mais: str += "entre os 10 melhores - "; break;
            case entre_dez_menos: str += "entre os 10 piores - ";
        }
    }
    str += "trama [" + classificação_trama + "] - direção [" + classificação_direção
        + "] - atuação [" + classificação_atuação
        + "] - " + formatarDataHora(data_hora.toString());
    return str;
}
```

O método toStringFull da entidade Avaliação utiliza os métodos toStringFull das classes Amigo, Filme e Ator, bem como, o método buscarAtoresFilme da entidade secundária Atuação.

O método buscarAtoresFilme, da entidade secundária Atuação, retorna o array de atores associados a um dados filme.

```
    public static Ator[] buscarAtoresFilme (int sequencial_filme) {
    String sql = "SELECT Sequencial, AtorId FROM Atuações WHERE FilmeId = ?";
    ResultSet lista_resultados = null;
    ArrayList<Ator> visões = new ArrayList();
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial_filme);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            visões.add(Ator.buscarAtor(lista_resultados.getString("AtorId")));
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    return visões.toArray(new Ator[visões.size()]);
}
```

O método toStringFull da classe Amigo:

```
public String toStringFull() {
    String str = nome;
    if (apelido != null) str += "[" + apelido + "]";
    str += " - " + sexo + " - " + estadoCivil + " - " + cidade;
    return str;
}
```

O método toStringFull da classe Filme, é sobreposto pelos respectivos métodos das subclasses.

```
public String toStringFull() { return ano + " - " + título +  " [" + gênero + "]"; }
```

O método toStringFull da subclasse FilmeCompanhiaCinematográfica:

```
public String toStringFull() {
    String str = super.toStringFull() + " - Oscars: ";
    if (oscar_melhor_filme) str += "Filme";
    if (oscarMelhorDiretor != null) str += " - Diretor [" + oscarMelhorDiretor + "]";
    if (oscarMelhorAtor != null) str += " - Ator [: " + oscarMelhorAtor + "]";
    if (oscarMelhorAtriz != null) str += " - Atriz : " + oscarMelhorAtriz + "]";
    return str;
}
```

O método toStringFull da subclasse FilmeProvedoraStreaming:

```
public String toStringFull() {
    String str = super.toStringFull() + " - " + provedora;
    if (produção == Produção.série) str += " - " + total_episódios + " episódios" ;
    return str;
}
```

O método toStringFull da classe Ator:

```
public String toStringFull() {
    String str = nome + " [" + ano_nascimento + "] - sexo: " + sexo;
    if (ganhador_oscar) str +=  " - oscar";
    return str;
}
```

No método estático pesquisarAvaliações, da classe Avaliação, o string correspondente ao comando SQL tem as seguintes características: (a) define mnemônicos para as três tabelas que utiliza; (b) em sua condição WHERE são incorporadas restrições associadas às duas chaves estrangeiras da tabela Avaliações, comparadas com as respectivas chaves das tabelas Amigo e Filme; e (c) são incorporadas condicionalmente (pois os filtros são opcionais) as restrições associadas aos filtros informados pelo usuário. De forma similar, os parâmetros do comando SQL, representados por interrogações, são substituídos condicionalmente pelos filtros informados pelos usuários. A numeração de cada interrogação, associada condicionalmente a um dado filtro, é caracterizada por um contador que vai sendo incrementado, a medida que um dado filtro é associado.

```
public static ArrayList<Avaliação> pesquisarAvaliações
    (Sexo sexo_amigo, EstadoCivil estado_civil_amigo,
    Gênero gênero_filme, int ano_filme, char oscar_melhor_filme, Produção produção_filme,
    Classificação classificação_trama, Preferência preferência, Timestamp data_mínima) {
    String sql = "SELECT Am.Nome, Am.Sexo, Am.EstadoCivil, F.Sequencial, F.Gênero, F.Ano,"
        + " Av.Sequencial, Av.ClassificaçãoTrama, Av.Preferência, Av.DataHora"
        + " FROM Amigos Am, Filmes F, Avaliações Av"
        + " WHERE Av.AmigoId = Am.Nome AND Av.FilmeId = F.Sequencial";
    if (sexo_amigo != null) sql += " AND Am.Sexo = ?";
    if (estado_civil_amigo != null) sql += " AND Am.EstadoCivil = ?";
    if (gênero_filme != null) sql += " AND F.Gênero = ?";
    if (ano_filme > -1) sql += " AND F.Ano >= ?";
    if (classificação_trama != null) sql += " AND Av.ClassificaçãoTrama = ?";
    if (preferência != null) sql += " AND Av.Preferência = ?";
    if (data_mínima != null) sql += " AND Av.DataHora >= ?";
    sql += " ORDER BY Av.Sequencial";
    ResultSet lista_resultados = null;
    ArrayList<Avaliação> avaliações_selecionadas = new ArrayList();
    int index = 0;
    int sequencial_filme = -1;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        if (sexo_amigo != null) comando.setInt(++index, sexo_amigo.ordinal());
        if (estado_civil_amigo != null) comando.setInt(++index, estado_civil_amigo.ordinal());
        if (gênero_filme != null) comando.setInt(++index, gênero_filme.ordinal());
        if (ano_filme > -1) comando.setInt(++index, ano_filme);
        if (classificação_trama != null) comando.setInt(++index, classificação_trama.ordinal());
        if (preferência != null) comando.setInt(++index, preferência.ordinal());
        if (data_mínima != null) comando.setTimestamp(++index, data_mínima);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            Avaliação avaliação_pesquisada
                 = Avaliação.buscarAvaliação(lista_resultados.getInt(7));
            sequencial_filme = lista_resultados.getInt(4);
            if (!atoresFilmeAtendemFiltros (sequencial_filme, ano_nascimento_mínimo_ator,
                ator_ganhador_oscar, todos_atores_filme)) continue;
            if (oscar_melhor_filme != 'X') {
                if (isOkPesquisaEmFilmesCompanhiasCinematográficas
                   (sequencial_filme, oscar_melhor_filme))
                        avaliações_selecionadas.add(avaliação_pesquisada);
            } else if (produção_filme != null) {
                if (isOkPesquisaEmFilmesProvedorasStreaming(sequencial_filme, produção_filme))
                    avaliações_selecionadas.add(avaliação_pesquisada);
            } else avaliações_selecionadas.add(avaliação_pesquisada);
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    return avaliações_selecionadas;
}
```

O método estático pesquisarAvaliações utiliza três métodos estáticos auxiliares.

O método auxiliar atoresFilmeAtendemFiltros, da classe Avaliação, verifica se os atores (todos ou pelo menos um, conforme o flag todos_atores_filme) de um filme, atendem os filtros dos atores. No projeto de referência desta disciplina, o relacionamento múltiplo envolve as entidades: Filme e Ator. A entidade principal do relacionamento múltiplo (Filme) é a superclasse de outras duas entidades: FilmeCompanhiaCinematográfica e FilmeProvedoraStreaming.

```
private static boolean atoresFilmeAtendemFiltros
    (int sequencial_filme, int ano_nascimento_mínimo_ator,
    char ator_ganhador_oscar, boolean todos_atores_filme){
    Ator[] atores_filme = Atuação.buscarAtoresFilme(sequencial_filme);
    int total_atores_não_atendem_filtros = 0;
    for (Ator ator : atores_filme) {
        if (((ano_nascimento_mínimo_ator != -1)
              && (ator.getAnoNascimento() < ano_nascimento_mínimo_ator))
            || ((ator_ganhador_oscar == 'T') && !ator.isGanhadorOscar())
            || ((ator_ganhador_oscar == 'F') && ator.isGanhadorOscar())) {
            total_atores_não_atendem_filtros++;
            if (todos_atores_filme) return false;
        }
    }
    if (total_atores_não_atendem_filtros == 0) return true;
    if ((todos_atores_filme) || (total_atores_não_atendem_filtros == atores_filme.length))
        return false;
    return true;
}
```

O método auxiliar isOkPesquisaEmFilmesCompanhiasCinematográficas, da classe Avaliação, verifica se o filtro da subclasse FilmeCompanhiaCinematográfica é atendido.

```
private static boolean isOkPesquisaEmFilmesCompanhiasCinematográficas
    (int sequencial_filme, char oscar_melhor_filme) {
    boolean pesquisa_ok = false;
    String sql = "SELECT * FROM FilmesCompanhiasCinematográficas WHERE FilmeId = ?";
    if (oscar_melhor_filme != 'X') sql += " AND OscarMelhorFilme = ?";
    ResultSet lista_resultados = null;
    int index = 1;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial_filme);
        switch(oscar_melhor_filme) {
            case 'T': comando.setBoolean(++index, true); break;
            case 'F': comando.setBoolean(++index, false);
        }
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) pesquisa_ok = true;
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) { exceção_sql.printStackTrace (); }
    return pesquisa_ok;
}
```

O método auxiliar isOkPesquisaEmFilmesProvedorasStreaming, da classe Avaliação, verifica se o filtro da subclasse FilmeProvedoraStreaming é atendido.

```
private static boolean isOkPesquisaEmFilmesProvedorasStreaming
    (int sequencial_filme, Produção produção_filme) {
    boolean pesquisa_ok = false;
    String sql = "SELECT * FROM FilmesProvedorasStreaming WHERE FilmeId = ?";
    if (produção_filme != null) sql += " AND Produção = ?";
    ResultSet lista_resultados = null;
    int index = 1;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial_filme);
        if (produção_filme != null) comando.setInt(++index, produção_filme.ordinal());
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) pesquisa_ok = true;
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    return pesquisa_ok;
}
```

Caso a classe secundária do relacionamento múltiplo (Ator) for a superclasse das duas outras subclasses, o método atoresFilmeAtendemFiltros, devem receber também os filtros das subclasses como parâmetros, e controlar a chamada dos outros dois métodos auxiliares para testar os filtros das subclasses. Neste caso, para cada ator do filme, seriam testados tanto os filtros do ator, como os filtros das subclasses da superclasse Ator. Na subseção seguinte, são comentadas as alterações que seriam necessárias para esta situação.

2.1.1 - Alterações para Projetos cuja Superclasse é a Secundária do Relacionamento Múltiplo

No projeto deste tutorial, o relacionamento múltiplo é: Filme [n:n] Ator e a superclasse é a classe principal do relacionamento múltiplo Filme com as subclasses FilmeCompanhiaCinematográfica e FilmeProvedoraStream.

No entanto, a entidade Ator, secundária do relacionamento múltiplo, poderia ser a superclasse. Para comentar as alterações decorrentes dessa situação, suponha que a superclasse Ator com as subclasses AtorConsagrado e AtorIniciante.

Neste caso, haverá necessidade de alterar trechos dos métodos pesquisarAvaliações e atoresFilmeAtendemFiltros.

Em vez de receber os filtros das subclasses de Filme, oscar_melhor_filme e produção_filme, o método pesquisarAvaliações passa a receber como argumentos os filtros das subclasses de Ator : ator_consagrado_veterano e ator_iniciante_protagonista_filme. No método pesquisarAvaliações, são implementadas duas mudanças principais:

- a chamada do método atoresFilmeAtendemFiltros passará também como argumentos, os filtros das subclasses: ator_consagrado_veterano e ator_iniciante_protagonista_filme;
- e os trechos seguintes que testam os filtros das subclasses de Filme serão removidos.

O código do método atoresFilmeAtendemFiltros é alterado para:

```
private static boolean atoresFilmeAtendemFiltros (int sequencial_filme,
    int ano_nascimento_mínimo_ator, char ator_veterano, char ator_consagrado_veterano,
    char ator_iniciante_protagonista_filme,  boolean todos_atores_filme){
     Ator[] atores_filme = Atuação.buscarAtoresFilme(sequencial_filme);
     int total_atores_não_atendem_filtros = 0;
     for (Ator ator : atores_filme) {
          String nome_ator = ator.getNome()
          if (((ano_nascimento_mínimo_ator != -1)
                && (ator.getAnoNascimento() < ano_nascimento_mínimo_ator))
              || ((ator_ganhador_oscar == 'T') && !ator.isGanhadorOscar())
              || ((ator_ganhador_oscar == 'F') && ator.isGanhadorOscar())
              || !isOkPesquisaEmAtoresFilmeCompanhiasCinematográficas(nome_ator,
                 ator_consagrado_veterano)
              || !isOkPesquisaEmAtoresFilmeProvedorasStreaming(nome_ator,
                 ator_iniciante_protagonista_filme)) {
              total_atores_não_atendem_filtros++;
              if (todos_atores_filme) return false;
          }
     }
     if (total_atores_não_atendem_filtros == 0) return true;
     if (todos_atores_filme || (total_atores_não_atendem_filtros == atores_filme.length))
        return false;
     return true;
}
```

Obviamente, será necessário adaptar os métodos que testam a pesquisa para os filtros das subclasses de Filme para testar os filtros das subclasses de Ator, considerando as tabelas com as subclasses de Ator : AtoresConsagrados e AtoresIniciantes.

2.2 - Os Tratamentos de Eventos de Limpeza

O método do tratador de eventos limparFiltros é ilustrado a seguir.

```
private void limparFiltros(java.awt.event.ActionEvent evt) {
    sexo_amigoButtonGroup.clearSelection();
    estado_civil_amigoButtonGroup.clearSelection();
    gêneros_filmesComboBox.setSelectedIndex(-1);
    ano_mínimo_filmeTextField.setText("");
    ano_nascimento_mínimo_atorTextField.setText("");
    ator_ganhador_oscarButtonGroup.clearSelection();
    todos_atores_filmeCheckBox.setSelected(false);
    avaliação_classificação_tramaComboBox.setSelectedIndex(-1);
    preferênciaButtonGroup.clearSelection();
    data_mínima_avaliaçãoTextField.setText("");
    filtros_filme_companhia_cinematográficaPainel.limparFiltros();
    filtros_filme_provedora_streamingPainel.limparFiltros();
}
```

O método do tratador de eventos limparPesquisa é ilustrado a seguir.

```
private void limparPesquisa(java.awt.event.ActionEvent evt) {
    pesquisasTextArea.setText("");
}
```